Yazılım Testi: Öncelik Sırası

ÖFY

September 2026

Özet

Test kurulumunun sırası üç terimli bir ölçütle belirlenir: hata sınıfının bu sistemde yaşanmış olması, kurulum süresi ve koşma süresi. İlk terim ağır basar; yalnızca kurulum ve koşma maliyeti sıfır olan denetimler ölçütten muaftır ve sürekli açık kalır. Regresyon testi yalnızca yaşanmış hatayı korur. Henüz yaşanmamış hata, tek tek hatalar yerine değişmezlerin yazılması ve bunların hem test anında rastgele girdiyle hem çalışma anında gerçek girdiyle denetlenmesiyle yakalanır. Değişmezler mevcut davranışı korur, yeni özelliği korumaz; bu nedenle her özellik kendi kuralıyla teslim edilir ve kural koddan önce yazılıp kırmızı görülür. Mutasyon testi test takımının kör olduğu yeri, varyant analizi ise bulunan hatanın henüz tetiklenmemiş örneklerini gösterir. Deploy anında ve kopan istekte kullanıcının zarar görmemesi tek bir mekanizmaya değil; nazik kapanış, hazırlık sinyali, idempotency, geri çekilmeli yeniden deneme ve iş kırasından oluşan bir ailenin birlikte kurulmasına bağlıdır.

Giriş

Yazılım testinin bilinen türleri yüzü aşar ve bir ekibin hepsini aynı anda kurması mümkün değildir. Hangi testin önce kurulacağına dair bir sıra olmadığında iki hata tekrarlanır: kolay kurulan ama hiç hata yakalamayan denetimler çoğalır, gerçekten yaşanmış hata sınıfları ise korunmasız kalır.

Bu sıranın dayandığı birkaç terim şöyle tanımlanır. **Test**, kodun belirli bir girdi karşısında beklenen davranışı gösterip göstermediğini otomatik olarak denetleyen programdır. **Regresyon testi** (*regression test*), daha önce yaşanmış bir hatanın geri dönmediğini doğrulayan testtir. **Değişmez** (*invariant*), hiçbir girdi ve koşulda bozulmaması gereken kuraldır. **Fixture**, testin girdi olarak kullandığı sabit örnek veridir; bir transkript, bir JSON yanıtı ya da kısa bir video dosyası olabilir.

Yaygın kabul, test sayısı arttıkça güvenliğin de arttığıdır. Bu kabul iki noktada yanılır. Birincisi, testlerin büyük çoğunluğu regresyon testiyse takım yalnızca geçmişe bakar. İkincisi, hiç kırmızı görülmemiş bir test kuralı denetlemediği hâlde yeşil kalabilir; sayı artar, koruma artmaz.

Sıra, TypeScript arka ucu ve ön yüzü, Python medya hattı ve yer yer Go servisleri olan, dış yapay zekâ sağlayıcılarına ve nesne depolamaya bağımlı bir web sistemi için kurulmuştur. Biçimsel doğrulama, mobil ve donanım testleri kapsam dışıdır.

Öncelik Ölçütü

Ölçüt, öncelik sırasıyla üç terimden oluşur: hata sınıfı bu sistemde yaşanmış mı; kurulum ne kadar sürüyor; koşma ne kadar sürüyor. İlk terim ağır basar. Kurulumu yarım saat süren ama hatası dün yaşanmış bir test, sıfır maliyetli ama bu sistemde hiç hata yakalamamış bir denetimin önünde durur.

Ölçütün iki istisnası vardır. Birincisi birinci gruptur: kurulum ve koşma maliyeti sıfırsa yaşanmışlık aranmaz ve denetim sürekli açık kalır, çünkü sıfır maliyetli bekçiyi kapalı tutmanın da bir maliyeti vardır. İkincisi yeni özelliğin kendi kuralıdır: yeni özelliğin henüz yaşanmış hatası olamaz, ama kuralın maliyeti özellik yazılırken ödendiği için her özellikte uygulanır. Bu ikinci istisna Yazım Zamanı bölümünde gerekçelendirilir.

Gruplar

1. Sıfır Maliyet, Sürekli Açık (Ölçütten Muaf)

- Tip denetimi
- Lint
- Bütünlük kontrolü (lockfile ve `npm ci`)
- Bağımlılık açık taraması
- Lisans denetimi
- Fitness function (satır limiti, katman ayrımı)
- Yapılandırma ve ortam (açılışta ortam değişkeni şeması)
- Race detector (yalnızca Go; mevcut testler `-race` bayrağıyla koşar. TypeScript ve Python'da karşılığı yoktur, o taraflar için kapsam dışıdır)

2. Mevcut Kodu Sabitleme (İlk Hafta)

- Flakiness takibi (`retries` kapalı, liste açık)
- Dosya bölme (characterization kaydı)
- Düzeltme öncesi kırmızı birim testi
- Regresyon testi
- Sözleşme
- Fixture parser testi
- Şema testi
- Bozuk fixture
- Giriş doğrulama
- Aşamalar arası veri sözleşmesi (Whisper segmenti ile cümle aynı şey değildir; SOT hatası bu sözleşme yazılmadığı için yaşanmıştır)
- Çıkış değişmezleri (test anında; aynı kuralların çalışma anında da koşması için bkz. Yazım Zamanı)

3. Girdi Tasarımı ve Hat Bütünlüğü (İlk Döngü)

- Sınır değer analizi
- Eşdeğerlik sınıfları
- Karar tablosu
- Durum geçişi
- Hata yolu testi
- Kütüphane hatasının yüzeye çıkması (bağımlılığın attığı hata yutulmadan çağırana ve kullanıcıya ulaşıyor mu)
- İdempotency
- Yeniden deneme (*retry with backoff*; yalnızca idempotent istekler tekrar denenir)
- Eşzamanlılık birim testi (kredi defteri ve meşgul sayacı yarışları)
- Kaynak temizliği
- Önbellek doğruluğu
- Property-based test (`fast-check`, `hypothesis`)
- Parite (TypeScript ve Python ayrıştırıcıları aynı fixture'ı okur)
- Metamorfik test (fixture ile)
- Ağ: birim ve bileşen katmanı (`msw`, `nock`, `httptest` ile gecikme, kopma, yarım cevap)
- Ağ: model tarafı (sahte sağlayıcı ile zaman aşımı, 429, yarım stream, tek ücret)
- Determinizm
- Hat bütünlüğü (sayım, yarım kalma)
- Girdi sınır testleri (boş, bozuk, çok büyük)
- Bozuk medya korpusu
- Zip-bomb ve zip-slip
- Karakter işkencesi
- Altın veri seti
- Saat ve saat dilimi
- Süre dolması

4. Kullanıcı Akışları ve Canlı Sağlık (İkinci Döngü)

- Kullanıcı hikâyelerinin çıkarılması
- Kabul testi
- Uçtan uca test (yapay zekâsız)
- Smoke testi
- Sanity testi
- Sistem testi
- Keşif testi
- Kullanıcı hatası (çift tıklama, geri tuşu, yenileme)
- Sekme geçişinde durum kaybı (başka sekmeye gidip dönünce form ve ilerleme yerinde mi)
- Süren iş varken çıkış uyarısı (sekme kapatılırken ya da sayfadan ayrılırken)
- Sentetik izleme (watchdog)
- Gözlemlenebilirlik (hata yolunda log var mı, correlation ID, alarm çalıyor mu)
- Doküman testi

5. Gerçek Bağımlılıkla Test (Gerçek Veritabanı, Kuyruk, Depolama İster)

- Zamanlanmış işlerin çift tetiklenmesi (askıda publish süpürmesi; tek instance varsayımı)
- Uzun süren iş ve süreç yeniden başlaması (aynı sınıf)
- Entegrasyon
- Eşzamanlı yazma
- Yarıda kalan işlem
- Kuyruk teslim garantisi
- Yetki matrisi
- Kiracı izolasyonu
- Faturalama değişmezi
- Kimlik

- Ağ: yükleme tarafı (gerçek depolamaya yarım yükleme)
- Sıra boşluğu
- Migrasyon ve geri dönüş
- Dağıtım ve geri alma
- Nazik kapanış (*graceful shutdown*) ve bağlantı boşaltma
- Hazırlık sinyali (*readiness probe*; kapanışta önce trafik kesilir)
- Yedekten dönüş

6. Yavaş, Ücretli ya da Kırılgan (Gece Koşusu, Sürüm Öncesi)

- Uçtan uca test (tam)
- AI eval (kurallı)
- Metamorfik test (gerçek modelle)
- Mutasyon testi
- Benchmark
- Ağ: sınır katmanı (Toxiproxy, `netem`)
- Ağ: indirme tarafı ve iç ağ
- Ağ: sistem katmanı (tarayıcı ağ profili)
- Ağ: test matrisi
- Arıza enjeksiyonu
- Disk dolması
- Sızıntı testi (uzun koşuda bellek, dosya tanıtıcısı, çocuk süreç ve dinleyici sayısı sabit kalıyor mu)
- Nesne depolama görünürlüğü
- Canlı izleme (kayma)
- Görsel regresyon
- Erişilebilirlik
- Yerelleştirme
- Uyumluluk

- Yapılandırılmış keşif
- AI eval (yargıçlı)
- Kombinatoryal (pairwise) test
- Model tabanlı test
- Kullanılabilirlik
- Kapsama ölçümü

7. İlk Arıza Yaşanınca Açılır

- Kaynak sınırları
- Canlı migrasyon kilidi
- Ağ: canlı coğrafi izleme
- Performans ailesi (istisna: bir kez keşif amaçlı koşulur, hatta alınmaz; önceki load/stress koşusu bu istisnadır)
- Dayanıklılık ailesi
- Devre kesici (*circuit breaker*)
- Üretim trafiği ailesi
- Güvenlik ailesi
- İnsan ve organizasyon ailesi
- Regülasyon ailesi

Kapsam Dışı

- Biçimsel doğrulama, sembolik yürütme, mobil senaryolar, donanım testleri

Yazım Zamanı ve Çalışma Anı

Öncelik sırası hangi testin önce kurulacağını belirler; bir testin hangi işle birlikte yazılacağını ve hangi anda koşacağını belirlemez. Bu iki soru gruplardan bağımsızdır ve her değişiklikte yeniden cevaplanır.

Regresyon Testinin Sınırı

Regresyon testi bilinen hatayı korur, henüz yaşanmamış hatayı yakalamaz. Yarım cümlede kesilen bir soundbite için belirli bir paket üzerinde yazılmış test yalnızca o paketi korur. Aynı hata başka bir pakette farklı bir yoldan doğarsa test sessiz kalır.

Hata Yerine Kural

Henüz yaşanmamış hatayı yakalamanın yolu tek tek hataları değil, değişmezi yazmaktır. "Hiçbir pakette soundbite yarım cümlede kesilmez" cümlesi bir değişmezdir; "şu paketteki soundbite 12. saniyede kesilmez" cümlesi bir regresyon örneğidir. Değişmez iki ayrı anda denetlenir:

- **Test anında, üretilmiş girdiyle.** Özellik tabanlı test (*property-based testing*), değişmezi çok sayıda rastgele ve uç girdi üzerinde dener (Claessen ve Hughes, 2000). Boş transkript, sıfır saniyelik klip, iki dilin karıştığı okuma gibi kimsenin elle yazmayacağı durumlar bu yolla bulunur.
- **Çalışma anında, gerçek girdiyle.** Çalışma zamanı denetimi (*runtime assertion*), aynı değişmezi her gerçek render'ın sonunda yeniden kontrol eder. Kural bozulursa paket temiz olarak işaretlenmez. Bu denetim, testin hiç görmediği gerçek görüntüdeki hatayı yakalar.

Kuralın baştan yazılıp sürekli denetlenmesi fikri sözleşmeyle tasarım (*design by contract*) adıyla bilinir (Meyer, 1988). İkinci gruptaki "Çıkış değişmezleri" maddesi bu fikrin test yarısıdır; çalışma anı yarısı aynı kuralın üretim hattında da koşmasını gerektirir.

Yeni Özelliğin Kendi Kuralı

Değişmezler mevcut davranışı korur, yeni eklenen bir özelliği kendiliğinden korumaz. Yeni özellik eski bir kuralı bozarsa yakalanır. Kendi işini yanlış yaparsa yakalanmaz, çünkü ne yapması gerektiği hiçbir yerde yazılı değildir. Kuralı yazılmamış ve elle denenmemiş bir özelliğin çalıştığını bilen kimse yoktur.

Bu nedenle her özellik kendi kuralıyla birlikte teslim edilir. Lower third'e kişi adını basan bir özellik "ad, konuşan kişinin görüntüsü üstüne düşer" değişmeziyle birlikte gelir. Ekip pratiğinde bu, bitmiş sayılma tanımının (*Definition of Done*) bir maddesidir: testi olmayan iş bitmemiştir. Bu düzende elle test hata aramak için yapılmaz, yalnızca özelliğin istenen davranış olup olmadığını onaylamak için yapılır.

Yazım Sırası

Kuralın koddan önce mi sonra mı yazıldığı, testin güvenilirliğini belirler.

Teste dayalı geliştirme (TDD). Önce test yazılır ve kırmızı olduğu görülür, ardından kod yazılır ve test yeşile döner (Beck, 2002). Kırmızı başlangıç, testin gerçekten kuralı denetlediğinin kanıtıdır.

Sonradan test (*test-after* ya da *test-last*). Önce kod yazılır, test sonra eklenir. Risk, testin her durumda geçmesidir: hiç kırmızı görülmemiş bir test, kuralı denetlemediği hâlde yeşil kalabilir.

Davranışa dayalı geliştirme (BDD). Test, düz dilde *Given / When / Then* kalıbıyla yazılır ve koda bağlanır (North, 2006). Böylece kuralın doğru kural olup olmadığına yazılımcı olmayan taraf da karar verebilir.

```
Given  muhabir okuması ve iki soundbite içeren bir paket
When   render alındığında
Then   hiçbir soundbite yarım cümlede kesilmemiş olmalı
```

TDD kodun doğruluğunu sorar, BDD ürünün istenen davranışı yapıp yapmadığını sorar. Tercih edilen sıra, kuralın koddan önce yazılıp kırmızı görülmesidir. Bu, ikinci gruptaki "Düzeltme öncesi kırmızı birim testi" maddesinin yeni özelliklere genişletilmiş hâlidir.

Testin Körlüğünü Ölçmek

Mutasyon testi (*mutation testing*), kodu bilerek bozar (`<` yerine `<=` yazmak, bir satırı silmek) ve test takımının bunu fark edip etmediğine bakar (DeMillo, Lipton ve Sayward, 1978). Fark edilmeyen bir bozulma, o bölgede ileride doğacak hatanın da yakalanmayacağını gösterir. Altıncı gruptaki madde, bu yönüyle gelecekteki hatanın nerede kaçacağının haritasıdır.

Varyant Avı

Bir hata bulunduğunda yalnızca o örnek düzeltilirse aynı kusurun diğer örnekleri yerinde kalır. Varyant analizi (*variant analysis*), bulunan hatanın biçimini çıkarıp kodun geri kalanında aynı biçimi arar. SOT hatasının kökü Whisper segmentinin cümle sayılmasıydı; aynı varsayım segmenti kullanan her aşamada henüz patlamamış bir hata olarak durur.

Kapanış ve Kopan İstek

Bir sunucu kopyası yenisiyle değiştirilirken ya da bir istek yolda koptuğunda kullanıcının zarar görmemesi, birbirini tamamlayan bir kavram ailesiyle sağlanır. Ailenin bir kısmı deploy anını, kalanı kopan isteğin sonucunu ele alır.

Deploy Anı

Nazik kapanış (*graceful shutdown*). Kapanma sinyali (`SIGTERM`) alan süreç hemen ölmez; yeni iş kabul etmeyi bırakır, süren işleri bitirir ya da güvenli bir noktada bırakır, sonra çıkar. Nazik kapanışı olmayan bir süreç her deploy'da o an süren render'ı ve yüklemeyi yarıda keser.

Bağlantı boşaltma (*connection draining*). Nazik kapanışın içindeki adımdır: yeni bağlantı alınmaz, açık olanların tamamlanması beklenir.

Kesintisiz deploy (*zero-downtime deploy*, *rolling update*). Yeni kopya ayağa kalkıp sağlık kontrolünden geçmeden eski kopya kapatılmaz. Platform bunu sağlasa bile eski kopyanın nazik kapanması uygulamanın sorumluluğudur.

Sağlık ve hazırlık sinyali (*health check*, *readiness probe*). Süreç "hazırım" ve "artık istek gönderme" sinyallerini ayrı ayrı verir. Kapanış başladığında önce hazırlık sinyali kapatılır, yük dengeleyici trafiği keser, ardından boşaltma başlar.

Kopan İsteğin Sonucu

İdempotency. Aynı istek iki kez geldiğinde iş bir kez yapılır. Kopan bir isteğin tekrar denenebilmesinin ön koşuludur; idempotent olmayan bir isteği tekrar denemek çifte ücret ya da çifte yayın üretir. Bu nedenle okuma istekleri tekrar denenir, değiştiren istekler ancak idempotency anahtarı taşıyorsa denenir.

Geri çekilmeli yeniden deneme (*retry with backoff*). İstemci başarısız isteği artan aralıklarla birkaç kez tekrar dener. Sabit aralıklı ve sınırsız deneme, bozulan servise yük bindirerek bozulmayı derinleştirir.

İş kirası (*job lease*) ve en az bir kez işleme (*at-least-once processing*). Uzun iş, süresi kısa ve yenilenen bir kira ile tutulur. İşçi ölürse kira dolar ve iş başka bir işçiye geçer. Bu düzen işin kaybolmasını önler, ancak aynı işin iki kez koşabileceğini kabul eder; bu yüzden idempotency ile birlikte kurulur.

Devre kesici (*circuit breaker*). Arkadaki servis art arda hata verdiğinde istemci bir süre istek göndermeyi bırakır ve hemen hata döner (Nygard, 2007). İstekler bozuk servisin önünde yığılmaz, servis toparlanınca devre yeniden kapanır. Tek bir dış servise bağlı küçük bir sistemde yeniden deneme çoğunlukla yeterlidir; devre kesici bu nedenle yedinci gruptadır.

İstemci Tarafındaki Karşılıklar

Aynı fikir tarayıcıda da geçerlidir. Süren bir iş varken sekme kapatılmak istendiğinde kullanıcı uyarılır (`beforeunload`). Sekmeler arasında gidip gelindiğinde form ve ilerleme durumu kaybolmaz. Bağımlı bir kütüphanenin attığı hata yutulmaz, kullanıcıya anlaşılır bir mesaj olarak ulaşır. Hatanın yutulması, sistemin çalıştığı izlenimini verirken işin sessizce kaybolması demektir.

Sızıntı

Sızıntı (*leak*), bir kaynağın kullanıldıktan sonra geri bırakılmamasıdır. Bellek, dosya tanıtıcısı, çocuk süreç, olay dinleyicisi ve açık bağlantı bu kaynaklardandır. Kısa testte görünmez, çünkü her koşu az miktarda bırakır; günler süren üretim çalışmasında birikerek çöküşe yol açar. Bu nedenle sızıntı testi aynı işi çok sayıda tekrarlayıp kaynak sayısının sabit kalıp kalmadığını ölçer.

Sonuç

Öncelik ölçütü neyin kurulacağını, yazım kuralı neyin ne zaman yazılacağını, kapanış ailesi ise sistemin hangi anda kırıldığını cevaplar. Üçü birlikte okunduğunda test, hata çıktıkça eklenen bir yama listesi olmaktan çıkar ve üç eksenli bir düzene dönüşür: geçmişi koruyan regresyon testleri, geleceği koruyan değişmezler ve geçiş anlarını koruyan kapanış mekanizmaları.

Ölçütün temel sınırlılığı geçmişe bakmasıdır. Yaşanmışlık şartı kaynakları gerçek hatalara yönlendirir, ama tek başına uygulandığında sistemi yalnızca bir kez yanmış yerlerden korur. Yeni özelliğin kendi kuralı, mutasyon testi ve varyant analizi bu eğilimi dengeleyen üç araçtır; üçü de hatanın yaşanmasını beklemeden çalışır.

Değişmezlerin çalışma anında denetlenmesi bir maliyet doğurur. Her render sonunda koşan denetim süre ekler ve yanlış alarm üretebilir; hangi değişmezin yalnızca test anında, hangisinin her render'da koşacağı açık bir sorudur. Medya hattında rastgele girdi üretmek de pahalıdır: rastgele video üretilemez, ancak rastgele transkript, zaman damgası ve kesim listesi üretilebilir. Bu nedenle özellik tabanlı test en çok ayrıştırıcı ve zamanlama katmanında verim verir, piksel katmanında ise altın veri seti ve görsel regresyon yerinde kalır.

Kesişim Taraması

Öncelik sırası testlerin hangi sırayla kurulacağını belirler. Mevcut bir sistemin bugün nerede kırık olduğunu göstermez; bunun için ayrı bir tarama gerekir. Bu bölüm, sistemin parça parça değil, parçaların buluştuğu noktalardan tarandığı bir çalışmayı ve bu çalışmanın literatürdeki karşılıklarını anlatır. Notta bu yöntem **kesişim taraması** adıyla anılır.

Neden Bu Kadar Çok Hata Çıktı

Tarama yaklaşık elli bulgu üretti. Sayının yüksek olmasının nedeni sorulan sorunun türüdür. Mevcut testler her parçayı tek başına denetler. Tarama ise "şu olay, şu durumdayken gelirse ne olur" sorusunu sorar.

Kombinasyon Sorusu

"İptal çalışıyor mu?" yerine "iptal, render bitip yükleme sürerken gelirse ne olur?" diye soruldu. Hataların çoğu, tek başına doğru çalışan iki parçanın buluştuğu yerdeydi:

- İptal ile yayın
- Replay ile atlama kuralı
- İmza ile soundbite
- Altyazı ile isim bandı

Kapıların Karşılaştırılması

Zip, kova, LiveU ve sayfa kapıları aynı kuralı ayrı ayrı uyguluyordu ve zamanla birbirinden ayrışmışlardı. `.MTS` uzantısı bir kapıda kabul, diğerinde ret ediliyordu. "vo" adı bir kapıda adla, diğerinde ses kanalıyla seçiliyordu.

Adımlar Arasında Kesinti

İş, adımların arasında kesildi. "Paket bitti, kayıt yazılırken sunucu çöktü" gibi anlar normal testlerde hiç oluşmaz, çünkü test her adımı ayrı koşar.

Gerçek Veriyle Karşılaştırma

Sahte XML ile her şey geçiyordu, gerçek LiveU dosyası hatayı hemen gösterdi. Laptoptaki `ffmpeg` (8.1) ile sunucudaki Debian sürümünün farklı olması da aynı türden bir hatadır.

Varsayımların Yazıya Dökülmesi

"Tek kopya çalışır" kuralı vardı, ama kod bunu kendisi zorlamıyordu; kuralı yalnızca bir ayar satırı koruyordu. Beşinci gruptaki "Zamanlanmış işlerin çift tetiklenmesi" maddesi bu varsayımın test karşılığıdır.

Hataların Birikme Nedeni

- **Hızlı büyüme.** Sisteme dört giriş kapısı, redo, Edit, LiveU ve otomasyon eklendi. Her yeni özellik kendi başına test edildi, eskilerle buluştuğu yerler edilmedi.
- **Nadir tetiklenme.** Bu hatalar günlük kullanımda seyrek ortaya çıkar: iki redo aynı saniyede, deploy tam yükleme sırasında, iki kamerada aynı dosya adı. Bu yüzden şikâyet gelmeden birikirler.

Bulguların Ağırlığı

Elli bulgunun hepsi ağır değildir. Yayına ya da veriye gerçekten dokunanlar on beş ile yirmi arasındadır: kaybolan paket, silinen video, geçmeli ve anamorfik görüntü, kısılan kelime, yanlış kesilen redo. Geri kalanlar uç durum ya da iç tutarlılık sorunudur.

Bulguların çoğu kod okunarak ve sahte veriyle kanıtlandı. Gerçek ortamda kaçının gerçekten patlayacağı bilinmiyor. Bazıları, örneğin çok sayıda muhabir ya da binin üzerinde kullanıcı gerektirenler, hiç yaşanmayabilir.

Tarama büyük bir özellikten sonra ya da müşteriye çıkmadan önce tekrarlanmalıdır. Sonraki taramalar daha az bulgu üretir; ilk tarama her zaman en çoğunu bulur.

Yöntemin Karşılıkları

Kesişim taramasının tek bir adı yoktur, birkaç bilinen yöntemin karışımıdır:

Yapılan	Karşılığı
Durum ve olay eşleşmeleri (iptal, yükleme sırasında gelirse)	State-space exploration, model tabanlı analiz
İki kuralın buluştuğu yeri arama	Feature interaction analysis
Kapıları birbiriyle, laptopu sunucuyla karşılaştırma	Differential testing, environment parity
Adımlar arası kesinti	Fault injection, crash consistency
Varsayımın kodda zorlanıp zorlanmadığına bakma	Invariant / assumption analysis
Sahte veri yerine gerçek dosyayla deneme	Fixture ve üretim verisi doğrulaması

Bütüne en yakın tek ad STPA'dır (*System-Theoretic Process Analysis*; Leveson, 2011). FMEA her parçanın nasıl bozulacağını tek tek sorar. STPA ise parçalar sağlamken aralarındaki etkileşimden doğan hatalara bakar. Taramadaki bulguların çoğu bu türdendir.

İlgili Terimler

Tasarım Aşamasında Hata Aramak

FMEA (*Failure Mode and Effects Analysis*). Her parça için nasıl bozulacağı ve bozulunca ne olacağı sorulur. Risk puanı üç terimin çarpımıdır: etkinin ağırlığı, görülme sıklığı ve fark edilme güçlüğü. Sessizce biriken hatalarda üçüncü terim belirleyicidir.

FTA (*Fault Tree Analysis*). Kötü sonuçtan geriye gidilir: "Paket kayboldu; hangi yollardan olabilir?"

STPA. Parçalar arasındaki etkileşimden doğan hatalara bakar (Leveson, 2011).

HAZOP (*Hazard and Operability Study*). Sapma kelimeleri her adıma tek tek uygulanır (Kletz, 1999). Klasik liste şunlardır: yok, fazla, az, kısmen, ayrıca, ters, başka, erken, geç, önce, sonra. Örneğin: "LiveU parçası geç gelirse?"

Pre-mortem. "Altı ay sonra büyük bir hata çıktı; sebebi neydi?" diye geriye bakılarak düşünülür (Klein, 2007).

Tehdit modelleme (STRIDE). Aynı mantığın güvenlik tarafıdır (Shostack, 2014). Örneğin bir kiracının diğerinin muhabirlerini görebilmesi. Beşinci gruptaki "Kiracı izolasyonu" maddesi bunun test karşılığıdır.

Kombinasyonlarla Test

Kombinatoryal (*pairwise*) test. Her ayar çifti en az bir kez birlikte denenir. Örneğin altyazı açık/kapalı, isim bandı var/yok ve format seçenekleri. Yazılım hatalarının büyük kısmı az sayıda parametrenin etkileşiminden doğar (Kuhn, Wallace ve Gallo, 2004). Altıncı gruptadır.

Özellik etkileşimi (*feature interaction*). İki özelliğin birbirini bozmasıdır. Örneğin replay ile atlama kuralı.

Durum uzayı taraması ve model denetimi (*model checking*). Bütün durumlar ve geçişler çıkarılıp gezilir. TLA+ bu alanın en bilinen aracıdır; Amazon, S3 ve DynamoDB dahil birçok servisin tasarımını bununla doğrulamıştır (Newcombe ve diğ., 2015). Biçimsel doğrulama bu notta kapsam dışıdır. Hafif hâli olan model tabanlı test altıncı gruptadır.

Özellik tabanlı test (*property-based testing*). Örnek yerine kural yazılır, araç çok sayıda rastgele girdi üretir. Python'da `hypothesis`, JavaScript'te `fast-check` kullanılır. Üçüncü gruptadır.

Gerçek Dünyayı Bozarak Test

Arıza enjeksiyonu (*fault injection*). Ağ kesintisi, disk dolması ya da veritabanı hatası bilerek araya sokulur. Altıncı gruptadır.

Kaos mühendisliği (*chaos engineering*). Aynı işlemin canlı sistemde yapılmasıdır (Basiri ve diğ., 2016). En bilinen örneği Netflix'in Chaos Monkey aracıdır.

Çökme tutarlılığı (*crash consistency*). "Tam bu satırda elektrik kesilirse veri tutarlı kalır mı?" sorusudur. Beşinci gruptaki "Yarıda kalan işlem" maddesine karşılık gelir.

Jepsen testi. Dağıtık sistemlerde birden çok kopya ve ağ bölünmesi altında tutarlılığı sınar. Tek kopya kilidi bunun küçük bir hâlidir.

Karşılaştırarak Bulmak

Diferansiyel test (*differential testing*). Aynı girdi iki yoldan verilir, sonuçlar karşılaştırılır (McKeeman, 1998). Zip kapısı ile kova kapısının karşılaştırılması bu türdendir. Üçüncü gruptaki "Parite" maddesi de aynı fikre dayanır.

Metamorfik test. Girdi bilinen bir biçimde değiştirilir ve çıktının buna uygun değişip değişmediğine bakılır (Chen, Cheung ve Yiu, 1998). Örneğin klip sırası değişince toplam süre aynı kalmalıdır.

Ortam paritesi (*environment parity*). Geliştirme makinesi ile sunucunun aynı olup olmadığı denetlenir. `ffmpeg` sürüm farkı buna örnektir.

Testlerin Kendisini Sınamak

Mutasyon testi. Koda bilerek küçük hatalar konur ve testlerin bunları yakalayıp yakalamadığına bakılır. JavaScript tarafında Stryker kullanılır. Altıncı gruptadır.

Kapsama (*coverage*). Satırların, dalların ve yolların ne kadarının test edildiğini ölçer. Yüksek kapsama, kesişimlerin denendiği anlamına gelmez. Her satır ayrı ayrı koşmuş olabilir, ama iki satır aynı durumda hiç birlikte koşmamış olabilir.

Sonraki Tarama İçin Yöntem

Kesişim taramasına en yakın üç yöntem birlikte kullanılır:

- Etkileşimler için STPA
- Adım adım sapma için HAZOP
- Kapıların ve ortamların karşılaştırılması için diferansiyel test

HAZOP'un sapma kelimeleri her adıma sistemli uygulanırsa, ilk taramada atlanan köşeler de yakalanır.

Kaynaklar

- Beck, K. (2002). *Test-Driven Development: By Example*. Addison-Wesley.
- Claessen, K. ve Hughes, J. (2000). QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. *ICFP 2000*.
- DeMillo, R. A., Lipton, R. J. ve Sayward, F. G. (1978). Hints on Test Data Selection: Help for the Practicing Programmer. *IEEE Computer*.
- Meyer, B. (1988). *Object-Oriented Software Construction*. Prentice Hall.
- North, D. (2006). Introducing BDD. *Better Software*.
- Nygard, M. T. (2007). *Release It! Design and Deploy Production-Ready Software*. Pragmatic Bookshelf.